Dokumentation Labor ESP32C3

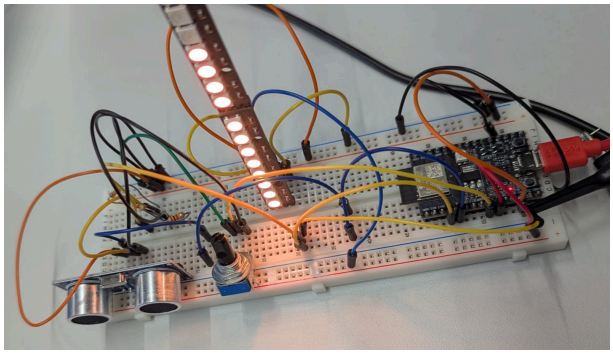

Figure 1: Fotografie der Schaltung mit Ultraschallsensor

Laboraufgabe 1

Um die LEDs anzusteuern haben wir uns für die Variante mit dem Bit-Banging entschieden. Hierfür wird der Dateneingang für die LEDs auf den `GPIO1` Pin geschalten.

Um die Bits in Serie auf den `GPIO1` Pin zu schicken gehen wir wie folgt vor: Wir lesen den Wert im `pixel` array als word und schieben ihn anschließend um 7 Bit nach links. So haben wir die 24 Bit, welche die Farbe eines einzelnen Neopixel steuern ganz links im Register. Anschließend lesen wir den Wert des MSB über ein bitweises AND aus und geben das entsprechende Spannungsmuster auf den `GPIO1` Pin. Das ganze geschieht durch eine Schleife mit Zählregister 24 mal.

Um das Spannungmuster zu erzeugen wird mit Assemblerbefehlen eine Warteschleife erzeugt. Die Anzahl der benötigten Wartezyklen haben wir experimentel bestimmt und am Oszilloskop bestätigt. Für eine präzise Wartezeit haben wir nach einer Schleife noch mit den benötigten `nop` befehlen aufgefüllt.

Die beschrieben Funktion wird in der Funktion `display(int factor)` aufgerufen, mit welcher eine Routine erzeugt werden kann, um die Farbe der Neopixel frequentiert anzupassen. Die Leuchtstärke der Neopixel kann über den Parameter `factor` gesetzt werden, in dem man diesem einen Wert zwischen 0 und 100 zuweist. Für die Funktion `testDisplay()` haben wir die `display`-Funktion verwendet, bei welcher mittels einer globalen Variable der gewünschte Pixel ein, und die restlichen ausgeschalten werden können.

Erweiternd haben wir ein Potientometer verwendet um über den adc des ESP32C3 die Leuchtstärke der Neopixel zu steuern. Hierfür wird der Wert des Potientometer am `GPIO0` Pin gelesen und über die `scaleSignal()` Funktion zu einem Faktor zwischen 1 und 100 gewandelt, welcher dann auf der `display`-Funktion übergeben werden kann.

Laboraufgabe 2 (15.04.2026)

Schaltung

Folgende Verschaltung wird genutzt:

- Ultraschallsensor: INIT auf `GPIO3` und ECHO auf `GPIO2` über Spannungsteiler

- LED: Dataeingang auf `GPIO1`
- Poti: Abgreifspannung auf `GPIO0`

Da `GPIO` Pins eine Spannung von 3.3V brauchen, wird vom ECHO Ausgang des USS ein Spannungsteiler aufgebaut. Aus $\frac{U_2}{U_1} = \frac{R_2}{R_1}$ und $U_1 + U_2 = 5V$ ergibt sich $\frac{R_2}{R_1} \approx 2$. Da wir drei 1kOhm Widerstände finden, bauen wir mit $R_2 = 2K$ mit zwei $1K$ in Reihe und $R_1 = 1K$ einen Spannungsteiler auf.

Software

Zuerst enablen wir die GPIO out Pins mit `*GPIO_ENABLE_REG = 0b00001010;` für die Neopixel LEDs und das `TRIG` Signal. Wir nutzen `*GPIO_OUT_REG |= (1 << 3);` um den Port auf `HIGH` zu setzen und das INIT-Signal für den USS zu senden. Wir warten dann mit einer For-Schleife die benötigte Zeit. Danach setzen wir mit `*GPIO_OUT_REG &= ~(1 << 3);` auf 0.

Wir nutzen `int isEchoing = (*GPIO_IN_REG & (1 << 2));` um den Status des Outsignals des USS zu prüfen. Um den Timer aufzusetzen, setzen wir zuerst den Divider auf 2 mit `TIMG_T0CONFIG_REG |= (1 << 13);` und den RST mit `TIMG_T0CONFIG_REG |= (1 << 12);`. Zu jeder Mainloop-Iteration resetten wir den Timer, sodass er mit 0 startet: `*TIMG_T0LOADLO_REG = 0;` und `*TIMG_T0LOAD_REG = 1;`, damit der Timer den Loadwert übernimmt. Wir starten bei Start des Echos den Timer `*TIMG_T0CONFIG_REG |= (1 << 31);`. Nachdem das Echo zuende ist, schalten wir den Timer aus `*TIMG_T0CONFIG_REG &= ~(1 << 31);` und latchen dann ans Hi und Lo Register: `TIMG_T0UPDATE_REG |= (1 << 31);`. Dann lesen wir `value = *TIMG_T0LO_REG`.

Wir berechnen den Abstandswert in mm wie folgt: $\text{time_us} = \text{sensor_value} * 40\text{MHz}^{-1} * 10^6 = \text{sensor_value} * \frac{1*1000000}{40*1000000} = \frac{\text{sensor_value}}{40} \rightarrow \text{dist_mm} = \frac{\text{time_us}}{58} * 10$ (58 aus dem Reference).

Um die LEDs anzusteuern implementieren wir eine Funktion `setPixelsFromDist()`. Wir berechnen einen Anteil $\text{factor} = \frac{\text{dist}}{\text{validRange}}$ wobei die valide Range zwischen 5 und 50 liegt. Damit berechnen wir den Index der LEDs, bis zu dem die LEDs leuchten sollen (siehe Code). Bei allen Berechnungen nutzen wir eine Multiplikation mit 100 und Division durch 100 zu späteren Zeitpunkten um ohne Floats rechnen zu können.

Wir wollen die LEDs von Grün zu Rot faden, je nachdem wie nah das Objekt ist. Wir nutzen den berechneten Faktor, um bei Ansteuerung der LEDs den Grünwert mit `factor` zu multiplizieren und den Rotwert mit `1 - factor`. Die Multiplikation erzeugt eine Abschwächung der einen Farbe während die andere bis Faktor 1 verstärkt wird.

Außerdem wollen wir implementieren, dass die jeweils letzte aktive LED beim Erscheinen nicht von Intensität 0 auf 100 springt, sondern durch Dimmen langsam eingeschalten wird. Wir erweitern unsere Funktion `setPixelsFromDist` um einen weiteren Intensitätsfaktor nach folgender Vorschrift:
`currentLedFade = 100 - (distance % ledStepDiff + 1) * 100 / ledStepDiff;` Die `ledStepDiff` ist das Delta, welches zu einer LED-Änderung auf dem Streifen führt (z.B. neue LED leuchtet). Durch die Berechnung mit Modulo erhalten wir den absoluten Wert, den wir bis zum nächsten "LED-Schritt" bereits erreicht haben. Wir addieren 1, da sonst Ergebnisse $0 \leq i \leq 27$ herauskommen, wir aber $1 \leq i \leq 28$ brauchen. Dieser Wert wird mit `ledStepDiff` dividiert, um den Intensitätsfaktor zu erhalten, mit dem der Binärwert des letzten Pixels wird.